

The Skynet Node Operational Grimoire

Distributed Spatial Architectures & Telemetry Control Planes

Lead Systems Administrator
`devsecfranklin@bitsmasher.net`

May 2026

Contents

Distributed Node Topology

The bitsmasher.net infrastructure is established on a decentralized, heterogeneous compute grid. Node assignment splits heavy I/O operations, JVM world-state execution, and on-device neural inferencing across optimized platforms.

1.1 Core Compute Grid Nodes

Chonk The primary gameplay execution engine. It hosts the Paper server and coordinates low-level coordinate mutations via custom bridge interfaces.

Skynet The local AI orchestration node. It processes neural pipeline coordination, NBT file manipulation, and runs the local spatial engines.

Stargate The high-throughput Model Context Protocol (MCP) hub and network routing plane.

Edge-T The physical network boundary agent handling telemetry interception.

1.2 System Metrics & Runtime baselines

To ensure predictable execution timings under heavy block modifications, the runtime states must align with the parameters defined in Table ??.

Table 1.1: Distributed Cluster Runtime Configurations

Node	Platform / Engine	Vessel Target	Network Port	Purpose
Chonk	Paper 26.1.5	Java 25	25565 / 25575	World Persistence
Skynet	Python 3.11 / venv	Debian Bookworm (rpi)	–	T2BM Orchestration
Stargate	Go / Python Daemons	Linux AMD64	5005	MCP / Tel. Hub
Edge-T	LiteRT / Mendel OS	i.MX 8M ARM64	–	Boundary Vision

Hardware Acceleration & Voxelization Mathematics

To eliminate high CPU thread wait times during procedural generation on Chonk, neural model execution is strictly offloaded to hardware edge accelerators.

2.1 Hardware Acceleration Framework

- **Vision Pipeline:** Quantized integer models run on the Edge-T node’s Coral Edge TPU. The host relies on the Mendel Development Tool (MDT) and SSH key authentication secured by `mdt-keymaster`.
- **Spatial Mapping:** Spatial engines on Skynet communicate directly with the local Hailo-8L NPU. Models must compile to the Hailo Executable Format (`.hef`) to enable direct memory access pipelines.

2.2 Voxel Mutation Calculation

Voxel placement calculations utilize a 3D coordinate transformation pipeline to map abstract model outputs to specific world delta regions.

Let $P_{\text{local}} = (x, y, z)^T$ be the local coordinate inside a structured schematic origin. The target world coordinate vector P_{world} on Chonk is computed using a translation offset vector $T = (T_x, T_y, T_z)^T$ and a rotation matrix $R(\theta)$:

$$P_{\text{world}} = R(\theta) \cdot P_{\text{local}} + T \quad (2.1)$$

Where $R(\theta)$ for a rotation of angle θ around the vertical Y -axis is defined by:

$$R_y(\theta) = \begin{pmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{pmatrix} \quad (2.2)$$

To avoid chunk desynchronization or locking during high block change throughput events, the processing latency (L) on the NPU must satisfy the performance inequality:

$$L_{\text{npu}} + \delta_{\text{rcon}} < \frac{1}{\text{TPS}} \quad \text{where } \text{TPS} \geq 20.0 \quad (2.3)$$

Schematic Standards & Voxel Generation

This chapter formalizes the architectural, stylistic, and structural technical standards for Minecraft schematics within the bitsmasher.net network, optimized specifically for the **v5 Industrial** era.

3.1 Project History & Voxelization Evolution

The bitsmasher.net cluster has evolved through three distinct architectural epochs, shifting from basic gameplay containers to distributed, hardware-accelerated spatial voxelization:

The Legacy Era (2012–2020): Characterized by manual constructions centered around the legacy Train Station. Navigation relied on armor-stand metadata tags, and administrative configurations were maintained via legacy FTB Ranks.

The Mycelial Era (2020–2025): Introduced early automated procedural generation. The aesthetic prioritized organic, mycelial growth networks. However, the lack of spatial safety checks resulted in silent deployment overlaps and frequent chunk-boundary corruption.

The v5 Industrial Voxel Era (2026–Present): Decommissioned organic “Void-Tech” logic in favor of high-density industrial greebling. Spatial math is offloaded to the local NVIDIA Jetson cluster (Edge-J) and validated via Skynet’s Hailo-8L NPU before injection, preserving a constant 20.0 TPS target.

3.2 v5 Industrial Aesthetic Specifications

Transitioning away from legacy biomorphic models, the v5 standard mandates highly structural, mechanically expressive block profiles.

3.2.1 Official Material Palette

All generative building algorithms must resolve abstract voxel requests using the following structural palette:

- **Primary Support Columns:** `minecraft:purpur_pillar` (configured as 3×3 fluted cores).
- **Structural Girders:** `minecraft:dark_prismarine` (simulating grid-iron assemblies).
- **Industrial Luminance:** `minecraft:pearlescent_froglight` (placed strictly at major structural intersections).
- **Temporal Detail Accents:** `minecraft:waxed_copper_block` and its oxidization variants.

- **Foundations & Retaining Walls:** `minecraft:polished_deepslate_bricks` and `minecraft:polished_andesite`.

3.2.2 Rigorous Design Laws

To guarantee depth, shadows, and parallax under live rendering engines (such as BlueMap), generative modules enforce three strict rules:

1. **Minimum Bounding Envelope:** Procedural generators must target a minimum bounding box of $20 \times 15 \times 25$ blocks. Anything smaller is rejected by the staging database.
2. **The Rule of Three:** Flat vertical walls are prohibited. Every exposed structural surface must contain three distinct topological depth steps:

$$\text{Surface Depth} = \mathbf{D}_{\text{Base Layer}} \cup \mathbf{D}_{\text{Structural Column}} \cup \mathbf{D}_{\text{Accent Girder}} \quad (3.1)$$

3. **Fluted Pillar Architecture:** Support columns must utilize a 3×3 horizontal footprint. Vertical pillar cores are surrounded by oriented, reversed stairs to create recessed shadowing effects.

3.3 The Emerald Mirror Pipeline

To modify the world state without locking the primary tick thread on Chonk, the cluster orchestrates a four-stage distributed transaction loop:

$$\text{Inference (Stargate)} \rightarrow \text{Refinement (Jetson Cluster)} \rightarrow \text{Validation (Skynet NPU)} \rightarrow \text{Injection (Chonk RCON)} \quad (3.2)$$

1. **Inference:** Stargate queries Ollama/Vertex AI with a localized spatial snapshot JSON. The model outputs a high-level creative instruction set.
2. **Refinement:** Abstract instructions are pushed via SSH/SCP to the CUDA-accelerated Jetson cluster (Edge-J01 through Edge-J04). Parallel kernels calculate voxel translations and write the target coordinates into binary format.
3. **Validation:** Skynet pulls the generated `.schem` file into memory, runs metadata integrity scripts, and cross-references its boundaries against protected coordinates using the Hailo-8L NPU.
4. **Injection:** Stargate transmits the verified changes to the live Paper console via RCON, utilizing asynchronous command throttling to maintain performance.

3.4 Metadata Structures & Schematic Classifications

To prevent generation algorithms from wiping out player-constructed infrastructure, schematics are divided into two classifications.

3.4.1 Classification Rules

- **Foundation Schematics:** Massive, solid skeletal “Anchors” containing permanent structural blocks. These represent the primary base layouts.
- **Delta Schematics:** High-density aesthetic overlays (greebles) containing a high proportion of `minecraft:air` blocks. These “wrap” around existing foundations dynamically.

3.4.2 Mandatory JSON Payload Schema

Every schematic transit must contain a companion metadata file. If this block fails validation, the downstream deployment queue on Chonk drops the request immediately:

```
1 {  
2   "build_id": "industrial_refinery_01",  
3   "schematic_type": "Foundation",  
4   "spatial_data": {  
5     "origin": {  
6       "x": -1024,  
7       "y": 63,  
8       "z": 1280  
9     },  
10    "dimensions": {  
11      "width": 20,  
12      "height": 15,  
13      "length": 25  
14    }  
15  }  
16 }
```

Listing 3.1: Mandatory Schematic Meta Payload

Distributed Time Synchronization

To guarantee chronological integrity across the bitsmasher.net distributed Minecraft cluster, system times must be synchronized with sub-millisecond precision. Accurate time tracking prevents write-collisions inside the neural storage databases and ensures exact sequence-locking of spatial telemetry events.

4.1 NTP Architecture & Topology

The cluster implements a secure, hardened NTP environment to distribute stable clock signals across all execution layers:

Authority Node `time.lab.bitsmasher.net` (Stratum 1 Local Time Source).

Daemon Implementation `ntpsec` (NTP Security Project).

Synchronization Mode Broadcast Client. Subordinate cluster nodes operate in broadcast client mode, listening passively for synchronization packets broadcasted on the local subnet to minimize polling-related network overhead.

Target Subnet Network `10.10.8.0/21`

4.2 Operational Security & Access Control

To mitigate potential NTP amplification attack vectors and prevent unauthorized time manipulation, strict access control lists are configured within the `ntpsec` context:

- **Global Restrict Rules:** Default IPv4 traffic is restricted to basic time exchange with zero query privileges:

```
1 restrict -4 default kod nomodify notrap nopeer noquery
2
```

- **Cluster Interrogation Access:** Nodes within the internal `10.10.8.0/21` range are granted interrogation privileges, allowing the Stargate Control Plane to dynamically audit peer clock health.
- **High-Speed Handshakes:** Authentication is explicitly disabled (`disable auth`) for local broadcast client operations to ensure high-speed, zero-latency handshakes within the trusted lab environment.

4.3 System Auditing & Metrics Loggers

Performance statistics are compiled daily inside the `/var/log/ntpstats/` namespace. System administrators must track three primary operational telemetry matrices:

1. **loopstats:** Tracks the local system clock stability, skew, and frequency drift over time.
2. **peerstats:** Logs upstream host metrics, capturing jitter, dispersion, and absolute round-trip delay.
3. **clockstats:** Records low-level hardware clock subsystem performance metrics.

4.4 Actionable Verification Commands

Synchronization states can be queried interactively from any node (such as Skynet, Edge-T, or the Jetson array) using standard control queries:

```
1 # Check peer synchronization status, offset, and jitter
2 ntpq -p
```

Listing 4.1: NTP Query Utility Verification

4.4.1 Thermal Drift Analysis

Frequency correction offsets are continuously appended to:

`/var/lib/ntp/ntp.drift`

Abrupt changes or high rates of change within this drift file are operational indicators of thermal fluctuations on local edge hardware, such as the Raspberry Pi 5 (Skynet) or Asus Tinker Edge T (Edge-T), directly impacting the physical crystal oscillators.

4.5 Impact on the Emerald Mirror Voxel Pipeline

Establishing a robust `ntpsec` broadcast timeline directly hardens structural deployment sequences:

$$\text{Edge-T Vision Sync} \iff \text{Stargate RCON Dispatch} \iff \text{Chonk Block Modification} \quad (4.1)$$

By lock-stepping the absolute system time of all nodes, transaction logs written to the Neural Data Vault (`registry.parquet`) use reliable, monotonic epoch millisecond timestamps. When a player builds in the “AI Testing Field,” the vision classification from Edge-T and the RCON injection from Stargate will be sequence-locked, preventing the “ghost building” errors common in high-latency distributed clusters.

Schematic & Injection Pipeline

The Text-to-Building (T2BM) generation pipeline converts spatial parameters into physical block data arrays inside the active world space.

5.1 Deployment Safety Checks

To protect the integrity of human-built architecture and designated cluster facilities, the system executes an automated test framework prior to dispatching any RCON payload:

```

1 # Navigate to repository root
2 cd /home/skynet/game-skynet-minecraft
3
4 # Initialize and execute pipeline validation suite
5 PYTHONPATH=./src ./venv/bin/python3 -m pytest \
6     test/test_schematic_metadata_integrity.py \
7     test/test_schematic_boundary_safety.py

```

Listing 5.1: Pre-deployment Verification Command Line

5.2 Validation Vectors

1. **Metadata Integrity Check:** Compares schematic headers against authorized NBT coordinate profiles to confirm structural bounds.
2. **Boundary Safety Check:** Intercepts injection payloads if coordinates intersect with restricted zones, such as the neural operation field or core transport lines.
3. **Payload Transit Auditing:** Tails `logs/test_core.log` and `logs/hailort.log` to verify that payloads are

5.3 Git LFS & World Synchronization

Binary files (`.schem`, `.mca`, and `.dat`) are handled strictly via Git LFS. Before performing commits, the world state must be frozen to prevent backing up partially written or corrupted regions:

```

1 # Freeze chunk flushing and lock auto-save routines on the live server
2 ./rcon.sh "save-off"
3 ./rcon.sh "save-all flush"
4
5 # Execute LFS synchronization
6 git add world/region/*.mca
7 git commit -m "sysadmin: world region state snapshot"
8 git push origin master

```

```
9  
10 # Restore server-side auto-save routines  
11 ./rcon.sh "save-on"
```

Listing 5.2: World Freeze and Sync Protocol

Operational Protocols & Telemetry

Our distributed nodes run asynchronous telemetry listeners to monitor server status and coordinate transport actions in real time.

6.1 Socket Configuration and Ports

Communication channels strictly adhere to the internal network matrix documented in Table ??.

Table 6.1: System Port Interface Matrix

Port	Protocol	Traffic Type	Service Endpoint
25565	TCP	Game Client Handshakes	Paper Minecraft Server
25575	TCP	Administrative Console	RCON Control Bridge
5005	TCP	Telemetry Events	CART_PASS Async Listener

6.2 Telemetry Event Processing

When a physical minecart triggers an accelerated detector rail array on Chonk, the telemetry sender encapsulates metadata regarding the transit and forwards it to Stargate on Port 5005.

Let E_{cart} be the telemetry event package. The serialized JSON envelope must strictly map to the following payload schema:

```

1 {
2   "event": "CART_PASS",
3   "node": "CHONK-01",
4   "data": {
5     "rail_id": "WASHINGTON_TERM_04",
6     "coordinates": {
7       "x": -1042,
8       "y": 64,
9       "z": 1289
10    },
11    "timestamp": 1778346300
12  }
13 }
```

Listing 6.1: CART_PASS Telemetry JSON Schema

6.3 Debugging Pipelines

If an AI agent confirms building generation but changes are not reflected in the live world, administrators must isolate the pipeline drop points:

$$\text{T2BM Output} \xrightarrow{\text{Validated}} \text{test_core.log} \xrightarrow{\text{Transited}} \text{hailort.log} \xrightarrow{\text{Injected}} \text{Chonk RCON} \quad (6.1)$$

By sequentially running the unit test targets via `pytest`, drop points can be identified and cleared before pipeline timeouts occur.